

Guía de ejercicios: ciclos y funciones en JavaScript

Fundamentos de programación

Reglas de la guía

- Solo puedes usar los ciclos `for` y `while`
- Cada ejercicio usa **mínimo dos funciones**. Al menos una debe recibir parámetros y devolver un resultado con `return`.
- Las funciones que calculan no usan `prompt` ni `console.log`: reciben datos y retornan el resultado.
- Lee los datos con `prompt` y muestra los resultados con `console.log`.
- `prompt` siempre devuelve texto. Convierte con `Number()` o `parseInt` o `parseFloat`: sin eso, `"5" + "3"` da `"53"`.
- Nombra las funciones con un verbo en camelCase: `calcularPromedio`, `esPrimo`.
- Las funciones sugeridas son el mínimo. Puedes crear más si lo necesitas.
- **Hacer la prueba de escritorio con los datos del ejemplo.**
- Si el enunciado pide 20 o 25 datos, prueba primero con 3 cambiando la cantidad temporalmente. Cuando funcione, vuelve al valor del enunciado.
- Todos los ejercicios deben llevar prueba de escritorio, a mano, en hojas o el cuaderno, para la entrega tomar foto de cada uno y armar un pdf llamado pruebas de escritorio y subirlo junto con los ejercicios en el repositorio.

En los ejemplos de ejecución, las líneas que empiezan con `[prompt]` muestran lo que el usuario escribe. Las demás son lo que imprime `console.log`.

¿Qué es una prueba de escritorio?

Una prueba de escritorio es ejecutar un algoritmo a mano, sin computador, anotando en una tabla cómo cambia el valor de cada variable paso a paso. Tú haces el trabajo del computador: lees cada línea, evalúas cada condición y registras cada cambio.

Sirve para tres cosas:

- Comprobar que la lógica da el resultado correcto.
- Encontrar errores típicos de los ciclos: que se repitan una vez de más, una vez de menos o nunca terminen.
- Entender qué hace un código que no escribiste tú.

Una tabla de prueba de escritorio tiene una columna por cada variable, una por cada condición importante y una para la salida. Cada fila representa un momento de la ejecución.

Ejemplo paso a paso

Problema: leer un número límite, contar cuántos múltiplos de 3 hay entre 1 y ese límite, y mostrar su suma.

```
function esMultiploDeTres(numero) {
  return numero % 3 === 0;
}

function sumarMultiplosDeTres(limite) {
  let suma = 0;
  let contador = 0;
  for (let i = 1; i <= limite; i++) {
    if (esMultiploDeTres(i)) {
      suma = suma + i;
      contador = contador + 1;
    }
  }
  console.log("Múltiplos de 3 encontrados: " + contador);
  console.log("Suma: " + suma);
}

let limite = Number(prompt("Ingrese el límite:"));
sumarMultiplosDeTres(limite);
```

Paso 1: identifica las variables

Lista todo lo que guarda un valor: el parámetro `limite`, las variables `suma` y `contador`, y la variable del ciclo `i`. También anota las condiciones que se evalúan: `i <= limite` y el resultado de `esMultiploDeTres(i)`.

Paso 2: elige los datos de entrada y calcula el resultado esperado

Usa un valor pequeño que puedas resolver mentalmente. Con `limite = 9`, los múltiplos de 3 son 3, 6 y 9. Resultado esperado: 3 múltiplos y suma 18. Anótalo, porque al final vas a comparar.

Paso 3: arma la tabla

Crea una columna por cada variable, una por cada condición y una para la salida. Agrega una columna "Paso" para numerar las filas.

Paso 4: registra los valores iniciales

Antes de entrar al ciclo, `limite` vale 9, `suma` vale 0 y `contador` vale 0. Esa es la fila 1. La variable `i` todavía no existe.

Paso 5: evalúa la condición del ciclo

Con `i = 1`, se pregunta `1 <= 9`. Es `true`, entonces se entra al ciclo. Si fuera `false`, el ciclo terminaría y pasarías a la línea que sigue después del `for`.

Paso 6: ejecuta el cuerpo del ciclo línea por línea

Cuando aparece una llamada a función, resuélvela aparte y anota solo lo que retorna. Por ejemplo, `esMultiploDeTres(3)` calcula `3 % 3`, que da 0; como `0 === 0` es `true`, la función retorna `true`. Entonces `suma` pasa de 0 a 3 y `contador` de 0 a 1.

Cuando la función retorna **false**, el **if** no se ejecuta y las variables quedan igual. Aun así se anota la fila, para dejar registro de esa vuelta.

Paso 7: ejecuta el incremento y repite

Se ejecuta **i++**, **i** pasa al siguiente valor y vuelves al paso 5. Repites hasta que la condición sea **false**.

Paso 8: registra la salida y compara

Cuando **i** llega a 10, la condición **10 <= 9** es **false** y el ciclo termina. Se ejecutan los **console.log**. La tabla completa queda así (en amarillo las vueltas donde cambian las variables):

Paso	limite	i	i <= limite	esMultiploDeTres(i)	suma	contador	Salida
1	9	-	-	-	0	0	-
2	9	1	true	false	0	0	-
3	9	2	true	false	0	0	-
4	9	3	true	true	3	1	-
5	9	4	true	false	3	1	-
6	9	5	true	false	3	1	-
7	9	6	true	true	9	2	-
8	9	7	true	false	9	2	-
9	9	8	true	false	9	2	-
10	9	9	true	true	18	3	-
11	9	10	false	-	18	3	Encontrados: 3 / Suma: 18

El programa imprime 3 múltiplos y suma 18. Coincide con el resultado esperado del paso 2, entonces la lógica es correcta.

¿Cómo detecta errores?

Supón que alguien escribió **i < limite** en lugar de **i <= limite**. Al hacer la prueba de escritorio, cuando **i** vale 9 la condición es **9 < 9**, que da **false**. El ciclo termina antes de revisar el 9, y la salida sería 2 múltiplos y suma 9.

Ese resultado no coincide con el esperado (3 y 18). Encontraste el error sin ejecutar el programa, y además sabes exactamente en qué línea está.

Bloque 1: Ciclo for

En estos ejercicios sabes cuántas veces se repite el proceso antes de empezar el ciclo, porque viene en el enunciado o lo lee el programa al inicio.

Ejercicio 1. Tabla de multiplicar

Qué prácticas: for con cantidad fija y una función con return llamada dentro del ciclo.

Enunciado

Leer un número y mostrar su tabla de multiplicar del 1 al 10. Cada línea debe mostrar el multiplicando, el multiplicador y el producto.

Ejemplo de ejecución

```
[prompt] Número: 7
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Funciones sugeridas

- `multiplicar(multiplicando, multiplicador)`: retorna el producto.
- `mostrarTabla(numero)`: recorre del 1 al 10 e imprime cada línea usando multiplicar.

Pista: Si ya sabes cuántas veces se repite antes de empezar, for es el ciclo indicado. ¿Desde qué valor arranca i y hasta cuál llega?

Ejercicio 2. Cubo y cuarta parte

Qué prácticas: for, funciones con return y acumulador.

Enunciado

Leer 10 números. Por cada número mostrar su cubo y su cuarta parte. Al terminar, mostrar la suma de todos los cubos.

Ejemplo de ejecución

```
[prompt] Número 1: 2
Número: 2 | Cubo: 8 | Cuarta parte: 0.5
[prompt] Número 2: 8
Número: 8 | Cubo: 512 | Cuarta parte: 2
... (8 números más)
Suma de los cubos: (según los 10 números)
```

Funciones sugeridas

- `calcularCubo(numero)`: retorna el número elevado al cubo.
- `calcularCuartaParte(numero)`: retorna la cuarta parte del número.

Pista: Primero haz que funcione con un solo número, sin ciclo. Cuando funcione, mete esa lógica dentro del for.

Ejercicio 3. Conversión de temperaturas

Qué prácticas: for con cantidad leída al inicio, acumulador y promedio.

Enunciado

Leer cuántos días se van a registrar. Luego leer la temperatura en grados centígrados de cada día y mostrar su equivalente en Kelvin y en Fahrenheit. Al final, mostrar la temperatura promedio en °C.

Fórmulas: $K = C + 273.15$ y $F = C * 9 / 5 + 32$.

Ejemplo de ejecución

```
[prompt] ¿Cuántos días?: 2
[prompt] Temperatura día 1 (°C): 25
Día 1: 25 °C = 298.15 K = 77 °F
[prompt] Temperatura día 2 (°C): 15
Día 2: 15 °C = 288.15 K = 59 °F
Promedio: 20 °C
```

Funciones sugeridas

- `convertirAKelvin(celsius)`: retorna la temperatura en Kelvin.
- `convertirAFahrenheit(celsius)`: retorna la temperatura en Fahrenheit.
- `calcularPromedio(suma, cantidad)`: retorna el promedio.

Pista: El promedio necesita dos datos: la suma y la cantidad. ¿En qué momento del ciclo se va sumando cada temperatura?

Ejercicio 4. Inversión con interés reinvertido

Qué prácticas: for y una variable que se actualiza con su propio valor en cada vuelta.

Enunciado

Una persona invierte su dinero en un banco que paga 2% de interés mensual. La ganancia de cada mes se reinvierte. Leer el capital inicial y la cantidad de meses. Mostrar el saldo al final de cada mes y la ganancia total.

Ejemplo de ejecución

```
[prompt] Capital inicial: 1000000
[prompt] Número de meses: 3
Mes 1: 1020000
Mes 2: 1040400
Mes 3: 1061208
Ganancia total: 61208
```

Funciones sugeridas

- `calcularInteres(saldo, porcentaje)`: retorna el interés que genera el saldo en un mes.
- `mostrarInversion(capital, meses)`: recorre los meses e imprime el saldo de cada uno.

Pista: El saldo del mes 2 se calcula sobre el saldo del mes 1, no sobre el capital inicial. ¿Qué variable debe cambiar en cada vuelta?

Bloque 2: Decisiones dentro del ciclo

Aquí el ciclo sigue siendo for, pero en cada vuelta el programa decide algo: qué contar, qué sumar o qué valor guardar.

Ejercicio 5. Positivos, negativos y ceros

Qué prácticas: contadores y condicionales usando el valor que retorna una función.

Enunciado

Leer 20 números e imprimir cuántos son positivos, cuántos negativos y cuántos son cero.

Ejemplo de ejecución

```
(probando con 5 números)
[prompt] Número 1: 4
[prompt] Número 2: -2
[prompt] Número 3: 0
[prompt] Número 4: 9
[prompt] Número 5: -7
Positivos: 2
Negativos: 2
Ceros: 1
```

Funciones sugeridas

- `clasificarNumero(numero)`: retorna el texto "positivo", "negativo" o "cero".
- `contarNumeros(cantidad)`: lee los números, usa `clasificarNumero` y muestra los tres conteos.

Pista: Necesitas tres contadores. ¿Se inicializan dentro o fuera del ciclo? Haz la prueba de escritorio con ambas opciones y compara.

Ejercicio 6. Salario semanal de obreros

Qué prácticas: condicional dentro del ciclo y acumulador.

Enunciado

Una empresa necesita calcular el salario semanal de N obreros. Leer N y luego las horas trabajadas de cada obrero.

Si el obrero trabaja 40 horas o menos, se le paga \$12.000 por hora. Si trabaja más de 40, se le pagan \$12.000 por cada una de las primeras 40 horas y \$15.000 por cada hora extra. Mostrar el salario de cada obrero y el total de la nómina.

Ejemplo de ejecución

```
[prompt] ¿Cuántos obreros?: 2
[prompt] Horas obrero 1: 38
Obrero 1: 456000
[prompt] Horas obrero 2: 45
Obrero 2: 555000
Total nómina: 1011000
```

Funciones sugeridas

- `calcularHorasExtra(horas)`: retorna cuántas horas pasan de 40 (0 si no hay).
- `calcularSalarioSemanal(horas)`: retorna el salario de la semana.

Pista: Las horas extra son solo las que pasan de 40. En el obrero 2, ¿cuántas horas se pagan a \$12.000 y cuántas a \$15.000?

Ejercicio 7. Expendio de naranjas

Qué prácticas: funciones que retornan booleanos, contador y acumulador.

Enunciado

Al cerrar un expendio de naranjas, 15 clientes que no han pagado reciben un 15% de descuento si compraron más de 10 kilos. Leer primero el precio por kilo y luego los kilos de cada cliente.

Mostrar cuánto paga cada cliente, el total que recibe la tienda y cuántos clientes obtuvieron descuento.

Ejemplo de ejecución

```
(probando con 3 clientes)
[prompt] Precio por kilo: 4000
[prompt] Kilos cliente 1: 5
Cliente 1 paga: 20000
[prompt] Kilos cliente 2: 12
Cliente 2 paga: 40800 (con descuento)
[prompt] Kilos cliente 3: 10
Cliente 3 paga: 40000
Total recaudado: 100800
Clientes con descuento: 1
```

Funciones sugeridas

- `tieneDescuento(kilos)`: retorna true o false.
- `calcularTotalCliente(kilos, precioKilo)`: retorna lo que paga el cliente, con o sin descuento.

Pista: "Más de 10 kilos" no incluye 10. Revisa en tu prueba de escritorio qué pasa con el cliente 3.

Ejercicio 8. Centro de verificación de automóviles

Qué prácticas: encontrar el mayor y el menor dentro de un ciclo.

Enunciado

Un centro de verificación quiere saber el promedio de puntos contaminantes de los primeros 25 automóviles que lleguen. También quiere saber los puntos del auto que menos contaminó y del que más contaminó.

Ejemplo de ejecución

```
(probando con 4 autos)
[prompt] Puntos auto 1: 120
[prompt] Puntos auto 2: 85
[prompt] Puntos auto 3: 200
[prompt] Puntos auto 4: 95
Promedio: 125
Menor contaminación: 85
Mayor contaminación: 200
```

Funciones sugeridas

- `obtenerMenor(a, b)`: retorna el menor de los dos números.
- `obtenerMayor(a, b)`: retorna el mayor de los dos números.
- `calcularPromedio(suma, cantidad)`: retorna el promedio.

Pista: Si inicializas la variable del menor en 0, ¿qué resultado da? Compruébalo en la prueba de escritorio antes de programar.

Ejercicio 9. Edades por género

Qué practicas: varios contadores y acumuladores, y evitar la división entre cero.

Enunciado

En un grupo de N aprendices se lee, uno por uno, el género (H o M) y la edad. Mostrar cuántos hombres y cuántas mujeres hay, el promedio de edad de los hombres, el de las mujeres y el de todo el grupo.

Si no hay hombres o no hay mujeres, mostrar un mensaje en lugar de calcular ese promedio.

Ejemplo de ejecución

```
[prompt] ¿Cuántas personas?: 4
[prompt] Género persona 1 (H/M): H
[prompt] Edad persona 1: 20
[prompt] Género persona 2 (H/M): M
[prompt] Edad persona 2: 18
[prompt] Género persona 3 (H/M): M
[prompt] Edad persona 3: 22
[prompt] Género persona 4 (H/M): H
[prompt] Edad persona 4: 30
Hombres: 2 | Promedio de edad: 25
Mujeres: 2 | Promedio de edad: 20
Promedio del grupo: 22.5
```

Funciones sugeridas

- `esHombre(genero)`: retorna true si el género es "H".
- `calcularPromedio(suma, cantidad)`: retorna el promedio.

Pista: Haz una prueba de escritorio donde todas las personas sean mujeres. ¿Qué pasa al calcular el promedio de los hombres?

Bloque 3: Ciclo while

En estos ejercicios no sabes cuántas veces se va a repetir el proceso. El ciclo termina cuando ocurre algo: se ingresa un valor especial (centinela) o se cumple una condición.

Ejercicio 10. Conversión de dólares a pesos

Qué practicas: while con valor centinela.

Enunciado

Un listado de personas recibe una cantidad de dólares. Por cada persona mostrar el equivalente en pesos, sabiendo que el dólar cuesta \$3.550. El listado termina cuando se ingresa 0.

Al final, mostrar cuántas personas se atendieron y el total en pesos. El 0 no cuenta como persona.

Ejemplo de ejecución

```
[prompt] Dólares (0 para terminar): 100
Equivale a: 355000 pesos
[prompt] Dólares (0 para terminar): 50
Equivale a: 177500 pesos
[prompt] Dólares (0 para terminar): 0
Personas atendidas: 2
Total en pesos: 532500
```

Funciones sugeridas

- `convertirDolaresAPesos(dolares)`: retorna el valor en pesos.
- `procesarListado()`: lee las cantidades hasta el 0 y muestra el resumen.

Pista: La primera lectura tiene que ocurrir antes de que el while evalúe su condición. ¿Dónde va la segunda lectura?

Ejercicio 11. Adivina el número

Qué practicas: while con dos condiciones de salida y números aleatorios.

Enunciado

El programa genera un número secreto entre 1 y 100. El usuario tiene máximo 7 intentos para adivinarlo. Después de cada intento, el programa dice si el número secreto es mayor o menor.

El juego termina cuando el usuario adivina o se le acaban los intentos. Si adivina, mostrar en cuántos intentos lo logró. Si pierde, mostrar cuál era el número.

Para generar el número usa: `Math.floor(Math.random() * 100) + 1`.

Ejemplo de ejecución

```
(número secreto: 42)
[prompt] Intento 1: 50
El número secreto es menor
[prompt] Intento 2: 25
El número secreto es mayor
[prompt] Intento 3: 42
¡Adivinaste en 3 intentos!
```

Funciones sugeridas

- `generarNumeroSecreto(minimo, maximo)`: retorna un número aleatorio entre minimo y maximo.
- `evaluarIntento(intento, secreto)`: retorna "mayor", "menor" o "correcto".

Pista: El ciclo se detiene por dos razones distintas. La condición del while tiene que tener en cuenta las dos.

Ejercicio 12. Calificaciones con validación

Qué practicas: while con centinela y validación de datos.

Enunciado

Leer las calificaciones de un grupo (escala de 0.0 a 5.0) sin saber cuántas son. La lectura termina cuando se ingresa -1.

Si una nota está fuera del rango, mostrar "Nota inválida" y no tenerla en cuenta. Al final mostrar: cantidad de notas válidas, promedio, nota más baja y cuántos aprobaron (nota mayor o igual a 3.0).

Ejemplo de ejecución

```
[prompt] Nota (-1 para terminar): 4.5
[prompt] Nota (-1 para terminar): 2.8
[prompt] Nota (-1 para terminar): 7
Nota inválida, debe estar entre 0 y 5
[prompt] Nota (-1 para terminar): 3
[prompt] Nota (-1 para terminar): -1
Notas válidas: 3
Promedio: 3.43 (en consola sale con más decimales)
Nota más baja: 2.8
Aprobados: 2
```

Funciones sugeridas

- `esNotaValida(nota)`: retorna true si la nota está entre 0 y 5.
- `estaAprobado(nota)`: retorna true si la nota es mayor o igual a 3.

Pista: Una nota inválida no suma, no cuenta y no puede ser la más baja. ¿Qué debe mostrar el programa si el primer dato ingresado es -1?

Bloque 4: Ciclos anidados

Un ciclo dentro de otro. Por cada vuelta del ciclo externo, el ciclo interno se ejecuta completo.

Ejercicio 13. Club contra la obesidad

Qué prácticas: for dentro de for.

Enunciado

Cinco miembros de un club quieren saber cuánto subieron o bajaron de peso desde la última reunión. Por cada miembro se lee su peso anterior y luego se pesa en 10 básculas distintas para obtener un promedio.

Si el promedio es mayor que el peso anterior, imprimir "SUBIÓ" y los kilos. Si es menor, imprimir "BAJÓ" y los kilos. Si es igual, imprimir "SE MANTUVO". Los kilos siempre se muestran en positivo, sin usar `Math.abs`.

Ejemplo de ejecución

```
(probando con 2 miembros y 3 básculas)
Miembro 1
[prompt] Peso anterior: 80
[prompt] Báscula 1: 81
[prompt] Báscula 2: 82
[prompt] Báscula 3: 83
SUBIÓ 2 kg
Miembro 2
[prompt] Peso anterior: 70
[prompt] Báscula 1: 68
[prompt] Báscula 2: 69
[prompt] Báscula 3: 67
BAJÓ 2 kg
```

Funciones sugeridas

- `leerPromedioBasculas(cantidadBasculas)`: lee los pesos de las básculas y retorna el promedio.
- `obtenerValorAbsoluto(numero)`: retorna el número en positivo.
- `generarLetrero(diferencia)`: retorna "SUBIÓ", "BAJÓ" o "SE MANTUVO".

Pista: El promedio de las básculas es distinto para cada miembro. ¿En qué lugar debe volver a 0 la suma de los pesos?

Ejercicio 14. Caja de supermercado

Qué prácticas: while dentro de while.

Enunciado

Un cajero registra los precios de los artículos de cada cliente. Un precio de 0 cierra la cuenta de ese cliente y se muestra su total. Después se pregunta si hay otro cliente (S/N).

Al final del día mostrar: cuántos clientes se atendieron, el total cobrado y cuál cliente hizo la compra más alta (número de cliente y monto).

Ejemplo de ejecución

```
Cliente 1
[prompt] Precio (0 para cerrar la cuenta): 3500
[prompt] Precio (0 para cerrar la cuenta): 12000
[prompt] Precio (0 para cerrar la cuenta): 0
Total cliente 1: 15500
[prompt] ¿Hay otro cliente? (S/N): S
Cliente 2
[prompt] Precio (0 para cerrar la cuenta): 8000
[prompt] Precio (0 para cerrar la cuenta): 0
Total cliente 2: 8000
[prompt] ¿Hay otro cliente? (S/N): N
Clientes atendidos: 2
Total cobrado: 23500
Compra más alta: cliente 1 con 15500
```

Funciones sugeridas

- `atenderCliente(numeroCliente)`: lee los precios hasta el 0 y retorna el total del cliente.
- `hayOtroCliente()`: pregunta y retorna true o false.

Pista: El total del cliente vuelve a 0 con cada cliente nuevo; el total del día no. Además de la compra más alta, ¿qué otro dato necesitas guardar para mostrar el número del cliente?

Ejercicio 15. Números primos

Qué practicas: función con un ciclo interno, llamada dentro de otro ciclo.

Enunciado

Leer un número N. Mostrar todos los números primos entre 2 y N, y al final cuántos hay.

Un número es primo si solo es divisible entre 1 y entre sí mismo. Para saber si `n` es divisible entre `d` usa `n % d === 0`.

Ejemplo de ejecución

```
[prompt] Límite: 20
2
3
5
7
11
13
17
19
Cantidad de primos: 8
```

Funciones sugeridas

- `contarDivisores(numero)`: retorna cuántos divisores tiene el número.
- `esPrimo(numero)`: retorna true si el número es primo.

Pista: Un número primo tiene exactamente 2 divisores. Haz la prueba de escritorio de `esPrimo(6)` y de `esPrimo(7)` por separado antes de armar el ciclo principal.